

Full Project Timeline and Overview

Project Name: BLLP-AI Language Learning Platform

Project Type: AI-Powered Web Application

Development Methodology: Agile Scrum

Sprint Duration: 2 Weeks

Total Sprints: 6

Total Project Duration: 12 Weeks

1. Project Overview

The BLLP-AI Language Learning Platform is an AI-powered web application designed to help users learn multiple languages through structured lessons, interactive content, speech support, and user progress tracking. The platform allows users to create an account, sign in, enroll in language courses, access lesson materials, continue learning, and manage their enrolled courses.

The project was developed using an Agile Scrum approach. The full development process was divided into six sprints, with each sprint lasting two weeks. Each sprint focused on a specific set of software engineering goals, including planning, system design, implementation, testing, refactoring, content management, and final deployment preparation.

2. Project Objectives

The main objective of this project was to design and develop a functional language learning platform that supports multiple courses and provides a clean learning experience for users.

The project objectives included:

Creating a secure user authentication system.

Building a responsive dashboard for course discovery and enrollment.

Developing structured lesson pages for language learning.

Supporting course enrollment, continue, and drop features.

Saving and tracking user progress.

Moving lesson content from hardcoded files into external CSV files.

Preparing the platform for future AI integration.

Improving the user interface and overall user experience.

Testing and refining the full application flow.

3. Scope of the Project

The scope of the project included the design and development of a web-based language learning platform. The application focused on user account management, course management, lesson content delivery, and progress tracking.

The project included frontend development, backend integration, Firebase authentication, lesson page development, content organization, API routes, UI improvements, and testing.

The project did not include full production-level AI tutoring, payment processing, live teacher support, or mobile app development. These features can be considered future improvements.

4. Technology Stack

The project used modern web development technologies to support scalability, maintainability, and usability.

Frontend: Next.js, React, TypeScript, Tailwind CSS

Backend and Services: Firebase, API routes

Database and Storage: Firebase services and external CSV lesson files

UI Tools: shadcn/ui, Framer Motion

Version Control: GitHub

Deployment Platform: Vercel

5. Development Methodology

The project followed the Agile Scrum methodology. The work was divided into six sprints, and each sprint focused on a specific group of features and improvements. This allowed the team to build the project step by step, test features regularly, and make improvements throughout the development process.

Each sprint included planning, implementation, review, testing, and documentation. This approach helped reduce project risk and allowed the application to improve over time.

6. Project Timeline

Sprint 1 focused on project planning, requirement analysis, system setup, and early UI design.

Sprint 2 focused on authentication, basic page development, Firebase setup, and dashboard structure.

Sprint 3 focused on course enrollment, course management, and lesson page structure.

Sprint 4 focused on progress tracking, dashboard improvements, and user interface cleanup.

Sprint 5 focused on externalizing lesson content into CSV files and refactoring the lesson system.

Sprint 6 focused on final testing, bug fixing, UI polish, deployment preparation, and project completion.

7. Functional Requirements

Users should be able to create an account.

Users should be able to log in securely.

Users should be able to view available language courses.

Users should be able to enroll in a course.

Users should be able to continue an enrolled course.

Users should be able to drop an enrolled course.

Users should be able to access structured lesson pages.

Users should be able to view vocabulary, key sentences, dialogue, listening, speaking, and quiz sections.

The system should save and display user progress.

The system should load lesson content dynamically from organized content files.

8. Non-Functional Requirements

The application should be responsive and user-friendly.

The application should have a clean and simple interface.

The system should be maintainable and easy to update.

The system should support future language expansion.

The application should use secure authentication.

The codebase should be organized and reusable.

The application should be tested before final submission.

9. Major Deliverables

The major deliverables of the project included:

A working Next.js web application.

Firebase authentication integration.

A user dashboard.

Course cards and enrollment functionality.

Lesson pages with structured content sections.

Progress tracking logic.

External CSV lesson content structure.

Dynamic lesson API routes.

Final tested version of the application.

Project documentation for all six sprints.

10. Conclusion

The BLLP-AI project was completed through six structured Agile sprints. Each sprint added an important layer to the application, starting from planning and setup and ending with final testing and project completion. The final product is a functional language learning platform with authentication, course enrollment, lesson pages, progress tracking, and improved content management.